

Interface Function Library (Secondary Development Library)

User's Manual

Instruction Manual Version: V2.10

Update Date: 2023.12.14

CONTENTS

Chapter I Overview	1
Chapter II ZLG Compatible Function Library and Data Structures	2
2.1 Type Definition	2
2.1.1 Device Type	2
2.1.2 VCI_BOARD_INFO	2
2.1.3 VCI_CAN_OBJ	4
2.1.4 VCI_INIT_CONFIG	6
2.2 Function Description	9
2.2.1 VCI_OpenDevice	9
2.2.2 VCI_CloseDevice	10
2.2.3 VCI_InitCan	11
2.2.4 VCI_ReadBoardInfo	14
2.2.5 VCI_GetReceiveNum	15
2.2.6 VCI_ClearBuffer	16
2.2.7 VCI_StartCAN	17
2.2.8 VCI_ResetCAN	19
2.2.9 VCI_Transmit	20
2.2.10 VCI_Receive	22
Chapter III Other Functions and Data Structure Descriptions	24
3.1 Type Definition	24
3.1.1 VCI_FILTER_RECORD	24
3.2 Function Description	25
3.2.1 VCI_UsbDeviceReset	25
3.2.2 VCI_FindUsbDevice2	26
3.2.3 VCI_SetReference	27
Chapter IV Interface Library Functions	31

Chapter I Overview

Users who only utilize the USB-CAN bus interface adapter for CAN bus debugging can directly utilize the USB-CAN Tool software provided with the product to test the sending and receiving data.

If users intend to write software programs for their own products. Please read the following instructions carefully and refer to the ones we provide:

(1) C++Builder (2) C# (3) VC (4) VB (5) VB.NET (6) Delphi (7) LabVIEW (8) LabWindows/CVI (9) Matlab (10) QT (11) Python/Python-can, etc. Sample Code

Development library file: ControlCAN.lib, ControlCAN.DLL

VC platform function declaration file: ControlCAN.h

VB platform function declaration file: ControlCAN.bas

LabVIEW platform library function package module: ControlCAN.llb

Delphi platform function declaration file: ControlCAN.pas

Chapter II ZLG Compatible Function Library and Data Structures

2.1 Type Definition

2.1.1 Device Type

Type Definition	Typed Value	Description
VCI_USBCAN2	4	USBCAN-2A USBCAN-2C CANalyst-II Series MiniPCIe-CAN Automotive OBDII Special Edition

2.1.2 VCI_BOARD_INFO

The VCI_BOARD_INFO structure contains device information for the USB-CAN family of interface cards. The structure will be populated in the VCI_ReadBoardInfo function.

```
typedef struct _VCI_BOARD_INFO {
    USHORT    hw_Version;
    USHORT    fw_Version;
    USHORT    dr_Version;
    USHORT    in_Version;
    USHORT    irq_Num;
    BYTE      can_Num;
    CHAR      str_Serial_Num[20];
    CHAR      str_hw_Type[40];
    USHORT    Reserved[4];
} VCI_BOARD_INFO, *PVCBOARD_INFO;
```

member:

hw_Version

Hardware version number in hexadecimal. For example, 0x0100 means V1.00.

fw_Version

Firmware version number, expressed in hexadecimal. For example, 0x0100 means V1.00.

dr_Version

Driver version number in hexadecimal. For example, 0x0100 means V1.00.

in_Version

Interface library version number, in hexadecimal. For example, 0x0100 means V1.00.

irq_Num

Reserved parameter.

can_Num

Indicates how many CAN channels are available.

str_Serial_Num

Serial number of this card.

str_hw_Type

Hardware type, e.g. "USBCAN V1.00" (Note: including the string terminator '\0')

Reserved

Reserved by the system.

2.1.3 VCI_CAN_OBJ

Description

The VCI_CAN_OBJ structure is a CAN frame structure, i.e. 1 structure represents a frame data structure. It is used to transmit CAN message frames in the transmit function VCI_Transmit and the receive function VCI_Receive.

```
typedef struct _VCI_CAN_OBJ {
    UINT    ID;
    UINT    TimeStamp;
    BYTE    TimeFlag;
    BYTE    SendType;
    BYTE    RemoteFlag;
    BYTE    ExternFlag;
    BYTE    DataLen;
    BYTE    Data[8];
    BYTE    Reserved[3];
} VCI_CAN_OBJ, *PVCI_CAN_OBJ;
```

Member:

ID

Frame ID. 32-bit variable with right-aligned data format. For details, please refer to: "8. Annex 1: ID Alignment.pdf" description document.

TimeStamp

The time stamp at which a frame was received by the device. The time stamp starts from the power-on of the CAN card, and the timing unit is 0.1ms.

TimeFlag

If or not the time flag is used, TimeStamp is valid when it is 1. TimeFlag and TimeStamp are only meaningful when this frame is a received frame.

SendType

Send frame type.

If =0, it is a normal send (send failure will be retransmitted automatically,

retransmission timeout is 1 second, if not sent within 1 second, it will be canceled);

= 1 for single send (send only once, send failure, arbitration failure will not automatically retransmit, the bus only produces a frame data waveform);

Other values are invalid.

Caution:

When communicating with multiple nodes, be sure to force **SendType** to be set to 0, otherwise there will be a case of sending lost frames.

For single node communication, e.g., as a CAN signal generator that sends out signals at regular intervals, **SendType** can be set to 1.

RemoteFlag

Whether the frame is a remote frame. = 0 for data frames, = 1 for remote frames (empty data segment).

ExternFlag

If or not is an extended frame. =0 is a standard frame (11-bit ID), =1 is an extended frame (29-bit ID).

DataLen

Data length DLC (≤ 8), i.e. how many bytes are in the CAN frame Data. Bounds the valid bytes in Data[8] later.

Data[8]

Data of the CAN frame. Since CAN specifies a maximum of 8 bytes, a space of 8 bytes is reserved here, constrained by DataLen. If DataLen is defined as 3, i.e. Data[0], Data[1], Data[2] are valid.

Reserved

Reserved by the system.

2.1.4 VCI_INIT_CONFIG

The VCI_INIT_CONFIG structure defines the configuration for initializing the CAN. The structure will be populated in the VCI_InitCan function, i.e., this structure variable is populated before initialization.

```
typedef struct _INIT_CONFIG {
    DWORD    AccCode;
    DWORD    AccMask;
    DWORD    Reserved;
    UCHAR    Filter;
    UCHAR    Timing0;
    UCHAR    Timing1;
    UCHAR    Mode;
} VCI_INIT_CONFIG, *PVCI_INIT_CONFIG;
```

Membership:

AccCode

Acceptance Code: SJA1000 frame filtering acceptance code. It matches the "relevant bits" filtered by the mask code, and the frame can be received after all the matches are successful. Otherwise, the frame will not be received. See VCI_InitCAN for details.

AccMask

SJA1000 frame filtering mask. It is used to filter the received CAN frame ID, bit 0 is "relevant bit", bit 1 is "irrelevant bit". **It is recommended to set the mask code to 0xFFFFFFFF, i.e., all frames are received.**

Reserved

Reserved.

Filter

Filter mode, allowable setting is 1-3, please refer to section 2.2.3 Filter Mode Comparison Table for details.

Timing0

Baud rate timer 0 (BTR0). The setting value is shown in the table below.

Timing1

Baud rate timer 1 (BTR1). The setting values are shown in the table below.

Mode

Mode. =0 means normal mode (equivalent to normal node), =1 means listen-only mode (receive only, does not affect the bus), and =2 means self-receive mode (loopback mode).

Remarks:

For filter settings, please refer to: "9. Annex 2: CAN Parameter Settings.pdf" instruction document.

Timing0 and Timing1 are used to set the CAN baud rate, several common baud rates (sampling point 87.5%, SJW is 0) are set as follows:

CAN Baud	Timing0(BTR0)	Timing1(BTR1)
20 Kbps	0x18	0x1C
40 Kbps	0x87	0xFF
50 Kbps	0x09	0x1C
80 Kbps	0x83	0xFF
100 Kbps	0x04	0x1C
125 Kbps	0x03	0x1C
200 Kbps	0x81	0xFA
250 Kbps	0x01	0x1C
400 Kbps	0x80	0xFA
500 Kbps	0x00	0x1C
666 Kbps	0x80	0xB6
800 Kbps	0x00	0x16
1000 Kbps	0x00	0x14
33.33 Kbps	0x09	0x6F
66.66 Kbps	0x04	0x6F
83.33 Kbps	0x03	0x6F

Note:

1. When configuring the baud rate, users only need to follow the baud rate parameters given by SJA1000 (16MHz) to set up.
2. Conventional baud rate can be configured directly according to the value in the above table. For other unconventional baud rates, you can use the included baud rate detection tool to detect and get the corresponding baud rate parameters. Or use the baud rate calculator in the USB_CAN TOOL installation directory. (Refer to "6. Plug-in 2: Baud Rate Detection Tool User Manual.pdf")
3. This adapter uses the latest high speed CAN transceiver and no longer supports baud rates below 10K.

2.2 Function Description

2.2.1 VCI_OpenDevice

This function is used to open a device. Note that a device can only be opened once.

```
DWORD __stdcall VCI_OpenDevice(DWORD DevType,DWORD DevIndex,DWORD Reserved);
```

Parameters:

DevType

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

Reserved

Reserved parameter, usually 0.

Return Value:

The return value = 1, means the operation is successful; = 0 means the operation fails; = -1 means the USB-CAN device does not exist or the USB is dropped.

Example:

```
#include "ControlCan.h"

int nDeviceType = 4;          /* USBCAN-2A or USBCAN-2C or CANalyst-II */
int nDeviceInd = 0;          /* First equipment */

DWORD dwRel;

dwRel = VCI_OpenDevice(nDeviceType, nDeviceInd, 0);

if(dwRel != 1)
{
    MessageBox(_T("Failed to Open Device!"), _T("Warning"), MB_OK|MB_ICONQUESTION);

    return FALSE;
}
```

2.2.2 VCI_CloseDevice

This function is used to close the device.

```
DWORD __stdcall VCI_CloseDevice(DWORD DevType,DWORD DevIndex);
```

Parameters:

DevType

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on. Corresponds to the device that has been turned on.

Return value:

The return value = 1 means the operation is successful; = 0 means the operation fails; = -1 means the USB-CAN device does not exist or the USB is dropped.

Example:

```
#include "ControlCan.h"

int nDeviceType = 4;          /* USBCAN-2A or USBCAN-2C or CANalyst-II */
int nDeviceInd = 0;          /* First equipment */

DWORD dwRel;

dwRel = VCI_CloseDevice(nDeviceType, nDeviceInd);

if(dwRel != 1)
{
    MessageBox(_T("Failed to Shut Down the Device!"), _T("Warning"), MB_OK|MB_ICONQUESTION);
    return FALSE;
}
```

2.2.3 VCI_InitCan

This function is used to initialize the specified CAN channel. When there are multiple CAN channels, it needs to be called multiple times.

```
DWORD __stdcall VCI_InitCAN(DWORD DevType, DWORD DevIndex, DWORD CANIndex,
PVCV_INIT_CONFIG pInitConfig);
```

Parameters:

DevType

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

CANIndex

CAN channel index. The CAN channel number of the corresponding card, CAN1 is 0, CAN2 is 1.

pInitConfig

Initialization parameter structure.

List of parameter members:

Members	Functional Description
pInitConfig->AccCode	By AccCode and AccMask can jointly decide which messages can be accepted, both registers are set using the left alignment of ID, i.e., the highest bit (Bit31) of AccCode and AccMask is aligned with the highest bit of the ID value.
pInitConfig->AccMask	For more information about ID alignment, please refer to: "Appendix 1: Detailed Description of ID Alignment" description document. Example: If the value of AccCode is set to

Members	Functional Description
	0x24600000 (i.e., the result of a 21-bit left shift of 0x123) and the value of AccMask is set to 0x00000000, only messages with CAN message frame ID 0x123 can be accepted (the value of 0x00000000 for AccMask means that all bits are relevant). If the value of AccCode is set to 0x24600000 and the value of AccMask is set to 0x600000 (the result of shifting 0x03 to the left by 21 bits), only the messages with CAN information frame ID 0x120~0x123 can be accepted (the value of AccMask 0x600000 means that all the bits (bit2~bit10) are related bits except for bit0~bit1). bit10) are all relevant bits). Note: This filter setting example takes the standard frame as an example, the high 11 bits are valid; if it is an extended frame, the ID is 29 bits, the high 29 bits are valid for extended frame when AccCode and AccMask are set!(bit2~bit10)
pInitConfig->Reserved	Reservation
pInitConfig->Filter	Refer to the Filter Mode Comparison Table in this section for filter mode settings.
pInitConfig->Timing0	Baud rate T0 setting
pInitConfig->Timing1	Baud rate T1 setting
pInitConfig->Mode	Operating modes: 0-Normal operation 1-Listening only mode 2-Self-transmitting and self-receiving test mode

Filter Mode Comparison Table:

Numeric	Name	Description
1	Receive All Types	The filter filters both standard and extended frames!
2	Standard Frames Only	The filter only filters standard frames, extended frames will be filtered out directly.
3	Standard Frames Only	The filter only filters extended frames, standard frames will be filtered out directly.

Return Value:

The return value = 1 means the operation is successful; = 0 means the operation fails; = -1 means the USB-CAN device does not exist or the USB is dropped.

Example:

```
#include "ControlCan.h"

int nDeviceType = 4;      /* USBCAN-2A or USBCAN-2C or CANalyst-II */
int nDeviceInd = 0;      /* First Equipment */
int nCANInd = 0;         /* First Channel */

DWORD dwRel;

VCI_INIT_CONFIG vic;

dwRel = VCI_OpenDevice(nDeviceType, nDeviceInd, 0);

if(dwRel != 1)
{
    MessageBox(_T("Failed to Open Device!"), _T("Warning"), MB_OK|MB_ICONQUESTION);
    return FALSE;
}

Vic.AccCode=0x80000008;

vic.AccMask=0xFFFFFFFF;

vic.Filter=1;

vic.Timing0=0x00;

vic.Timing1=0x14;
```

```

vic.Mode=0;

dwRel = VCI_InitCAN(nDeviceType, nDeviceInd, nCANInd, &vic);

if(dwRel !=1)
{
    VCI_CloseDevice(nDeviceType, nDeviceInd);

    MessageBox(_T("Failed to initialize device!"), _T("Warning"), MB_OK|MB_ICONQUESTION);

    return FALSE;
}

```

2.2.4 VCI_ReadBoardInfo

This function is used to get the device information.

```

DWORD __stdcall VCI_ReadBoardInfo(DWORD DevType,DWORD
DevIndex,PVCI_BOARD_INFO pInfo);

```

Parameters:

DevType

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

pInfo

Pointer to the VCI_BOARD_INFO structure used to store device information.

Return Value:

The return value = 1 means the operation is successful; = 0 means the operation fails; = -1 means the USB-CAN device does not exist or the USB is dropped.

Example:

```

#include "ControlCan.h"

int  nDeviceType = 4;      /* USBCAN-2A or USBCAN-2C or CANalyst-II */
int  nDeviceInd = 0;      /* First Equipment */

VCI_BOARD_INFO vbi;

DWORD dwRel;

dwRel = VCI_ReadBoardInfo(nDeviceType, nDeviceInd, &vbi);

if(dwRel != 1)
{
    MessageBox(_T("Failed to get device information!"), _T("Warning"), MB_OK|MB_ICONQUESTION);

    return FALSE;
}

```

2.2.5 VCI_GetReceiveNum

This function is used to get the number of frames received but not yet read in the receive buffer of the specified CAN channel. The main purpose is to use with VCI_Receive, i.e., there is data in the buffer, and then receive it.

In practice, users can ignore this function and call VCI_Receive directly in a loop, which can save PC system resources and improve program efficiency.

```
DWORD __stdcall VCI_GetReceiveNum(DWORD DevType,DWORD DevIndex,DWORD CANIndex);
```

Parameters:

DevType

Device type. Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

CANIndex

CAN channel index. The CAN channel number of the corresponding card, CAN1 is 0, CAN2 is 1.

Return Value:

Returns the number of frames that have not yet been read, = -1 means that the USB-CAN device does not exist or the USB is dropped.

Example:

```
#include "ControlCan.h"

int  nDeviceType = 4;      /* USBCAN-2A or USBCAN-2C or CANalyst-II */
int  nDeviceInd = 0;       /* First Equipment */
int  nCANInd = 0;          /* First Channel */

DWORD dwRel;

dwRel = VCI_GetReceiveNum (nDeviceType, nDeviceInd, nCANInd);
```

2.2.6 VCI_ClearBuffer

This function is used to clear the buffer of the specified CAN channel. It is mainly used to clear the receive buffer data, and the transmit buffer data will be cleared at the same time.

In practice, users can ignore this function and call VCI_Receive directly in a loop, which will automatically clear the read data and return the buffer space, which can save PC system resources and improve program efficiency.

```
DWORD __stdcall VCI_ClearBuffer(DWORD DevType,DWORD DevIndex,DWORD CANIndex);
```

Parameters:

DevType

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

CANIndex

CAN channel index. The CAN channel number of the corresponding card, CAN1 is 0, CAN2 is 1.

Return Value:

The return value = 1, means the operation is successful; = 0 means the operation fails; = -1 means the USB-CAN device does not exist or the USB is dropped.

Example:

```
#include "ControlCan.h"

int nDeviceType = 4;    /* USBCAN-2A or USBCAN-2C or CANalyst-II */

int nDeviceInd = 0;     /* First Equipment */

int nCANInd = 0;        /* First Channel */

DWORD dwRel;

dwRel = VCI_ClearBuffer(nDeviceType, nDeviceInd, nCANInd);
```

2.2.7 VCI_StartCAN

This function is used to start one of the CAN channels of the CAN card. When there are multiple CAN channels, it needs to be called multiple times.

```
DWORD __stdcall VCI_StartCAN(DWORD DevType,DWORD DevIndex,DWORD CANIndex);
```

Parameters:

DevType

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

CANIndex

CAN channel index. The CAN channel number of the corresponding card, CAN1 is 0, CAN2 is 1.

Return Value:

The return value = 1, means the operation is successful; = 0 means the operation fails; = -1 means the USB-CAN device does not exist or the USB is dropped.

Example:

```
#include "ControlCan.h"

int  nDeviceType = 4;    /* USBCAN-2A or USBCAN-2C or CANalyst-II */

int  nDeviceInd = 0;     /* First Equipment */

int  nCANInd = 0;        /* First Channel */

DWORD dwRel;

VCI_INIT_CONFIG vic;

if(VCI_OpenDevice(nDeviceType, nDeviceInd, 0) != 1)

{

    MessageBox(_T("Failed to Open Device!"), _T("Warning"), MB_OK|MB_ICONQUESTION);

    return FALSE;

}

if(VCI_InitCAN(nDeviceType, nDeviceInd, nCANInd, &vic) != 1)

{

    VCI_CloseDevice(nDeviceType, nDeviceInd);

    MessageBox(_T("Failed to Initialize Device!"), _T("Warning"), MB_OK|MB_ICONQUESTION);

    return FALSE;

}

if(VCI_StartCAN(nDeviceType, nDeviceInd, nCANInd) != 1)

{

    VCI_CloseDevice(nDeviceType, nDeviceInd);

    MessageBox(_T("Failed to Start the Device!"), _T("Warning"), MB_OK|MB_ICONQUESTION);

    return FALSE;

}
```

2.2.8 VCI_ResetCAN

This function is used to reset the CAN, mainly used with VCI_StartCAN to restore the normal state of the CAN card without initialization. For example, this function can be called when the CAN card enters the bus shutdown state.

```
DWORD __stdcall VCI_ResetCAN(DWORD DevType,DWORD DevIndex,DWORD CANIndex);
```

Parameters:

DevType

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

CANIndex

CAN channel index. The CAN channel number of the corresponding card, CAN1 is 0, CAN2 is 1.

Return Value:

The return value = 1, means the operation is successful; = 0 means the operation fails; = -1 means the USB-CAN device does not exist or the USB is dropped.

Example:

```

#include "ControlCan.h"

int  nDeviceType = 4;      /* USBCAN-2A or USBCAN-2C or CANalyst-II */
int  nDeviceInd = 0;      /* First Equipment */
int  nCANInd = 0;         /* First Channel */

DWORD dwRel;

dwRel = VCI_ResetCAN(nDeviceType, nDeviceInd, nCANInd);

if(dwRel != 1)
{
    MessageBox(_T("Reset Failed!"), _T("Warning"), MB_OK|MB_ICONQUESTION);

    return FALSE;
}

```

2.2.9 VCI_Transmit.

Transmit function. The return value is the number of frames actually sent successfully.

```

DWORD __stdcall VCI_Transmit(DWORD DeviceType,DWORD DeviceInd,DWORD
CANInd,PVCI_CAN_OBJ pSend,DWORD Length);

```

Parameters:*DevType*

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

CANIndex

CAN channel index. The CAN channel number of the corresponding card, CAN1 is 0, CAN2 is 1.

pSend

First pointer to the VCI_CAN_OBJ array of the frame structure to be sent.

Length

Length of the frame structure array to be sent (number of frames to be sent). The maximum is 1000, it is recommended to set it to 1 and send a single frame at a time to improve the sending efficiency.

Return Value:

Returns the number of frames actually sent, = -1 indicates that the USB-CAN device does not exist or the USB is down.

Example:

```
#include "ControlCan.h"

int  nDeviceType = 4;      /* USBCAN-2A or USBCAN-2C or CANalyst-II */
int  nDeviceInd = 0;       /* First Equipment */
int  nCANInd = 0;          /* First Channel */

DWORD dwRel;

VCI_CAN_OBJ vco[48];

for(int i=0;i<48;i++)
{
    vco[i].ID = i;

    vco[i].RemoteFlag = 0;

    vco[i].ExternFlag = 0;

    vco[i].DataLen = 8;

    for(int j = 0;j<8;j++)
        vco.Data[j] = j;
}

dwRel = VCI_Transmit(nDeviceType, nDeviceInd, nCANInd, vco,48);
```

2.2.10 VCI_Receive

Receive function. This function reads data from the receive buffer of the specified device CAN channel.

```
DWORD __stdcall VCI_Receive(DWORD DevType, DWORD DevIndex, DWORD CANIndex,
PVCI_CAN_OBJ pReceive, ULONG Len, INT WaitTime);
```

Parameters:

DevType

Device type. See: Adapter Device Type Definition for details on the different product models.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter, then the index number of the device inserted after it is 1, and so on.

CANIndex

CAN channel index. The CAN channel number of the corresponding card, CAN1 is 0 and CAN2 is 1.

pReceive

First pointer to the VCI_CAN_OBJ array of frame structures to be used for receiving. Note: The actual size of the array must not be smaller than the len parameter below, otherwise memory read/write errors will occur.

Len

The length of the received frame structure array (the maximum number of frames to be received, the actual return value is less than or equal to this value). This value is the size of the storage space provided, the adapter has set up a receive buffer of about 2500 frames for each channel, and the user selects the appropriate receive array length between 1 and 2000 according to the needs of their own systems and working environments. Generally, the size of pReceive array and Len are both set to 2500, so that the data in the cache can be read out at once. vCI_Receive is called in a cyclic way, so that the data in the cache can be read out at the fastest speed.

WaitTime

Reserved parameter.

Return Value:

Returns the number of frames actually read, = -1 means USB-CAN device does not exist or USB is offline.

Example:

```
#include "ControlCan.h"

int  nDeviceType = 4;      /* USBCAN-2A or USBCAN-2C or CANalyst-II */
int  nDeviceInd = 0;       /* First Equipment*/
int  nCANInd = 0;          /* First Channel */

DWORD dwRel;

VCI_CAN_OBJ vco[2500];

dwRel = VCI_Receive(nDeviceType, nDeviceInd, nCANInd, vco,2500,0);

if(IRel > 0)
{
    ...                      /* Data Processing */
}

else if(IRel == -1)
{
    ...                      /* The USB-CAN device does not exist or the USB is dropped, you can call
                               VCI_CloseDevice and re-VCI_OpenDevice. so that you can achieve the
                               effect of hot-plugging of the USB-CAN device. */
}
```

Chapter III Other Functions and Data Structure Descriptions

This chapter describes other data types and function descriptions of the non-compatible Zhou Ligong (ZLG) interface library provided to the user in ControlCAN.dll, the interface function library of the USB-CAN adapter, with the purpose of expanding the functions of the USB-CAN adapter to give the best performance. If the ZLG-compatible mode is used for secondary development, please do not call these functions to avoid affecting compatibility.

3.1 Type Definition

3.1.1 VCI_FILTER_RECORD

Defines the filter range VCI_FILTER_RECORD structure for the filter. The structure is populated in the VCI_SetReference function.

```
typedef struct _VCI_FILTER_RECORD {
    DWORD    ExtFrame;
    DWORD    Start;
    DWORD    End;
} VCI_FILTER_RECORD,*PVCI_FILTER_RECORD;
```

Member:

ExtFrame

Filtered frame type flag, 1 means the frame to be filtered is an extended frame, 0 means the frame to be filtered is a standard frame.

Start

ID of the start frame of the filter range.

End

End frame ID of the filtering range.

3.2 Function Description

3.2.1 VCI_UsbDeviceReset

Reset the USB-CAN adapter, after reset you need to open the device again using VCI_OpenDevice. It is equivalent to plugging and unplugging the USB device once.

```
DWORD __stdcall VCI_UsbDeviceReset(DWORD DevType,DWORD DevIndex,DWORD Reserved);
```

Parameters:

DevType

The device type. See: adapter device type definition for details.

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, when there are more than one, the index number is incremented from 0.

Reserved

Reserved.

Return Value:

The return value = 1 means the operation is successful; = 0 means the operation fails; = -1 means the device does not exist.

Example:

```
#include "ControlCan.h"

int nDeviceType = 4;      /*USB-CAN2.0 */
int nDeviceInd = 0;      /* Device 0 */
int nCANInd = 0;

DWORD dwRel;

bRel = VCI_UsbDeviceReset(nDeviceType, nDeviceInd, 0);
```

3.2.2 VCI_FindUsbDevice2

When multiple USB-CANs are used on the same PC, this function can be used to find the current device and get the serial numbers of all devices. Up to 50 devices are supported.

```
DWORD __stdcall VCI_FindUsbDevice2(PVCI_BOARD_INFO pInfo);
```

Parameters:

pInfo

The first address of a structure array used to store information such as the device serial number. The array length is recommended to be defined as 50, e.g. VCI_BOARD_INFO pInfo[50].

Return Value:

Returns the number of USB-CAN adapters inserted in the computer.

Example:

```
#include "ControlCan.h"

CString ProductSn[50];
VCI_BOARD_INFO pInfo [50];
int num=VCI_FindUsbDevice2(pInfo);
CString strtemp,str;
for(int i=0;i<num;i++)
{
    str="";
    for(int j=0;j<20;j++)
    {
        strtemp.Format("%c", pInfo [i]. str_Serial_Num [j]);
        str+=strtemp;
    }
    ProductSn[i]="USBCAN-"+str;
}
```

The main role:

1, software and hardware binding: the user secondary development software, you can realize the software and hardware unique binding. By calling this function, you can get the serial number of the device and compare it with the serial number pre-set by the user's application program. Realize the unique binding and encryption of software and hardware. When a single device, you can also directly use VCI_ReadBoardInfo function to fill the str_Serial_Num parameter in the VCI_BOARD_INFO structure for encryption.

2、Multi-card with the same machine, the device index number and the serial number of a one-to-one match: multiple devices at the same time plugged into a computer, to operate the corresponding device, you need to know at this time the corresponding device index number of the device. By calling this function, the structure array pInfo in the corresponding structure in the element str_Serial_Num can save the serial number of all devices.

pInfo[0].str_Serial_Num, pInfo[1].str_Serial_Num, pInfo[2].str_Serial_Num device index numbers are 0, 1, 2, 3 in that order.

3.2.3 VCI_SetReference

This function is used to set the smart filter table.

```
DWORD __stdcall VCI_SetReference(21, DWORD DevIndex, DWORD CANIndex, DWORD
RefType, PVOID pData);
```

parameter:

DevIndex

Device index, for example, when there is only one USB-CAN adapter, the index number is 0, then insert another USB-CAN adapter then the index number of the device inserted after it is 1, and so on.

CANIndex

CAN channel index. The CAN channel number of the corresponding card, CAN1 is 0, CAN2 is 1.

RefType

Parameter type.

pData

Used to store the parameter about the first pointer of the data buffer address.

Return Value:

A value of 1 indicates a successful operation, 0 indicates a failed operation.

Remarks

VCI_SetReference function is mainly used to set the intelligent filter parameters. The PVOID parameter pData in the function has different meanings with different operations.

VCI_SetReference function, RefType=1, RefType=2, RefType=3 three values are mainly for intelligent filtering function.

Users can add filter table by RefType=1, and can add any filter range.

With RefType=2, you can enable the filter table.

With RefType=3, you can clear the filter table.

Caution; the filter table can only be written, not read, and should be enabled after adding the filter table. To modify the filter table, you can only clear the filter table and add it again. Filter table can not be saved when power down, it needs to be set and enabled every time after initialization.

RefType	pData	Functional Description
1	Pointer to VCI_FILTER_RECORD structure	The filter table populated with CAN filters (this function is called once for each record added) is used for frame reception filtering for these types of devices. It should be set with the VCI_SetReference function after calling the VCI_InitCAN function.
2	NULL,negligible	Filtering is initiated as set in the filter table and is enabled by a call. It should be set with the VCI_SetReference function after calling the VCI_InitCAN function
3	NULL,negligible	Clear Filtering

Example:

```
#include "ControlCan.h"

int nDeviceInd = 0; // Index number 0

int nCANInd = 0; // CAN0 channel

VCI_FILTER_RECORD FILTER[4];

FILTER[0].ExtFrame=0;FILTER[0].Start=0x0001;FILTER[0].End=0x0001;//Receive a
single standardized frame ID, ID=0x0001

FILTER[1].ExtFrame=0;FILTER[1].Start=0x0003;FILTER[1].End=0x0005;//Receive
standard frame ID range, ID=0x0003-0x0005

FILTER[2].ExtFrame=1;FILTER[2].Start=0x00000002;FILTER[2].End=0x00000002;//Rec
eive a single extended frame ID, ID=0x00000002

FILTER[3].ExtFrame=1;FILTER[3].Start=0x00000004;FILTER[3].End=0x00000007;//Rece
ive Extended Frame ID Range, ID=0x00000004-0x00000007


VCI_SetReference(21, nDeviceInd, nCANInd,3,0);// If smart filtering was originally
configured, you can first clear the original filter list,

VCI_SetReference(21, nDeviceInd, nCANInd,1,&FILTER[0]);// Add Smart Filter List
VCI_SetReference(21, nDeviceInd, nCANInd,1,&FILTER[1]);// Add Smart Filter List
VCI_SetReference(21, nDeviceInd, nCANInd,1,&FILTER[2]);// Add Smart Filter List
VCI_SetReference(21, nDeviceInd, nCANInd,1,&FILTER[3]);// Add Smart Filter List
VCI_SetReference(21, nDeviceInd, nCANInd,2,0);// Enable the above filter list
```


Chapter IV Interface Library Functions

In order to enrich the functions of the device, we provide some additional functions (the functions indicated by the green background in the following figure), these functions include: [VCI_FindUsbDevice2](#) 、 [VCI_UsbDeviceReset](#), these functions are not necessary to call when carrying out the secondary development, and you can realize all the functions of the USB-CAN adapter by ignoring these functions. functions of the USB-CAN adapter can be realized by ignoring these functions.

